Mini sistem za personalizovano učenje reči i fraza

Seminarski rad iz predmeta Algoritmi i strukture podataka

1 Uvod

U savremenim aplikacijama za učenje jezika sve veći značaj imaju sistemi koji se prilagođavaju korisniku i njegovom tempu učenja. Umesto statičnog prikaza sadržaja, moderni sistemi koriste algoritme i strukture podataka kako bi optimizovali proces memorisanja.

Cilj ovog rada je implementacija mini sistema za učenje reči i fraza stranog jezika, koji koristi više koncepata iz oblasti algoritama i struktura podataka. Sistem omogućava korisniku da unosi odgovore, a zatim adaptivno određuje koje reči treba ponavljati.

U okviru rada korišćeni su sledeći koncepti:

- Trie (prefiksno drvo)
- Rabin–Karp algoritam
- model učenja baziran na grafu
- Spaced Repetition princip

Opis sistema Sistem funkcioniše kao interaktivni kviz. Korisniku se prikazuje reč ili fraza na srpskom jeziku, a njegov zadatak je da unese odgovarajući prevod na engleski.

Na osnovu korisnikovog odgovora sistem:

- proverava tačnost
- ažurira statistiku učenja
- određuje sledeći termin ponavljanja
- bira narednu reč za testiranje

Podaci o rečima čuvaju se u JSON fajlu, dok se napredak korisnika čuva u posebnom fajlu, čime se omogućava trajno skladištenje stanja sistema.

2 Implementacija sistema

2.1 `Word.h`

U okviru sistema, osnovna jedinica podataka predstavlja struktura `Word`. Ova struktura modeluje jednu reč ili frazu, zajedno sa svim informacijama koje su potrebne za praćenje napretka korisnika.
Izgled struture koja implementira centralni objekat **Word** nalazi se ispod.

```
struct Word {
    std::string serbian;
    std::string english;

    int correctCount = 0;
    int wrongCount = 0;
    int masteryLevel = 0;
    int intervalDays = 1;
    int learningWeight = 5;
    std::time_t nextReviewTime = 0;

    Word() = default;

    Word(const std::string& sr, const std::string& en)
        : serbian(sr), english(en) {
        nextReviewTime = std::time(nullptr);
    }
};

inline void to_json(json& j, const Word& w) { }

inline void from_json(const json& j, Word& w) { }
```

2.1.1 Opis strukture

Struktura `Word` predstavlja centralni model podataka u sistemu. Svaki objekat ove strukture sadrži tekstualne podatke o reči ili frazi, kao i dodatne informacije koje omogućavaju personalizovano učenje.

Polja strukture imaju sledeće značenje:

- `serbian` – tekst reči ili fraze na srpskom jeziku

- `english` – odgovarajući prevod na engleski jezik
- `correctCount` – broj tačnih odgovora korisnika
- `wrongCount` – broj pogrešnih odgovora korisnika
- `masteryLevel` – nivo usvojenosti reči
- `intervalDays` – broj dana do sledećeg ponavljanja
- `learningWeight` – težina reči koja utiče na prioritet u učenju
- `nextReviewTime` – vremenski trenutak kada reč ponovo treba da se pojavi

2.1.2 Konstruktori

Struktura poseduje dva konstruktora:

- podrazumevani konstruktor `Word()` koji omogućava kreiranje praznog objekta
- konstruktor sa parametrima koji inicijalizuje srpski i engleski tekst

Prilikom kreiranja nove reči, vrednost `nextReviewTime` postavlja se na trenutno vreme, čime se omogućava da reč odmah bude dostupna za učenje.

2.1.3 JSON konverzija

Za potrebe čuvanja i učitavanja podataka koristi se biblioteka `nlohmann::json`. Definisane su dve funkcije:

- `to_json` – konvertuje objekat tipa `Word` u JSON zapis
- `from_json` – konvertuje JSON zapis u objekat tipa `Word`

Funkcija `to_json` eksplicitno navodi sva polja koja se čuvaju u fajlu. Sa druge strane, `from_json` koristi metodu `value`, koja omogućava postavljanje podrazumevanih vrednosti ukoliko određeno polje ne postoji u JSON zapisu.

Ovo je posebno važno jer početna baza reči ne mora sadržati sve informacije o napretku korisnika.

2.1.4 Uloga u sistemu

Struktura `Word` koristi se u svim ključnim modulima sistema:

- **Scheduler** koristi informacije o težini, greškama i vremenu ponavljanja
- **LearningGraph** ažurira nivo usvojenosti i težinu reči
- **Trie** koristi engleski tekst za pretragu
- **SuggestionHelper** koristi podatke za rangiranje predloga
- **GUI** prikazuje informacije korisniku
- **ProgressManager** čuva i učitava podatke iz fajlova

Na ovaj način struktura `Word` predstavlja osnovu celokupnog sistema i omogućava povezivanje svih njegovih komponenti.

2.2 `ProgressManager.h`

Modul `ProgressManager` zadužen je za upravljanje podacima sistema, uključujući učitavanje baze reči, čuvanje napretka korisnika i prikaz informacija o učenju. Potpis funkcija koje su implementirane u ovom modulu nalaze se ispod.

```
#ifndefPROGRESS_MANAGER_H
#define PROGRESS_MANAGER_H

#include <vector>
#include <string>
#include <fstream>
#include <iostream>
#include <filesystem>
#include <algorithm>
#include "Word.h"

class ProgressManager {
public:
    static std::vector<Word> loadWordsFromBase(const std::string&
        filename) { }

    static std::vector<Word> loadProgressOrBase(const std::string
       & baseFile,
```

```
                                                    const std::string
                                                       & progressFile)
                                                        { }

    static bool saveProgress(const std::string& filename, const
       std::vector<Word>& words){};

    static void printWords(const std::vector<Word>& words) { }

    static void printTopHardestWords(const std::vector<Word>&
       words, int limit = 5) { }
};

#endif
```

2.2.1 Uloga modula

Modul `ProgressManager` ima ključnu ulogu u sistemu jer omogućava trajno čuvanje i učitavanje podataka. Na taj način sistem može da pamti napredak korisnika između različitih pokretanja aplikacije.

2.2.2 Učitavanje baze reči

Funkcija `loadWordsFromBase` učitava početnu bazu reči iz JSON fajla.

Proces:

- otvara se fajl pomoću `ifstream`
- JSON sadržaj se parsira
- konvertuje se u vektor objekata tipa `Word`

U slučaju greške (npr. fajl ne postoji ili je neispravan), funkcija vraća prazan vektor.

2.2.3 Učitavanje progresa ili baze

Funkcija `loadProgressOrBase` omogućava fleksibilno pokretanje sistema.

Logika rada:

- ako postoji fajl sa progresom, učitava se on
- u suprotnom, učitava se početna baza reči

Na ovaj način korisnik može nastaviti učenje tamo gde je stao.

2.2.4 Čuvanje progresa

Funkcija `saveProgress` zapisuje trenutni status sistema u JSON fajl.

Proces:

- vektor reči se konvertuje u JSON
- zapisuje se u fajl sa indentacijom radi čitljivosti

Ova funkcija se poziva nakon svakog odgovora korisnika.

2.2.5 Prikaz podataka

Funkcija `printWords` koristi se za prikaz svih reči i njihovih atributa u konzoli. Ovo je korisno za testiranje i debugovanje sistema.

2.2.6 Pronalaženje najtežih reči

Funkcija `printTopHardestWords` sortira reči prema težini i prikazuje najproblematičnije.

Kriterijumi sortiranja:

- najveća težina ima prioritet
- zatim broj grešaka
- zatim niži nivo usvojenosti

Ova funkcija omogućava identifikaciju reči koje zahtevaju dodatnu pažnju tokom učenja. Zbog kriterijuma sortiranja, neprovežbane reči imaju najniži prioritet.

2.2.7 Povezanost sa ostatkom sistema

Modul `ProgressManager` koristi se u više delova sistema:

- **GUI** koristi ovaj modul za učitavanje i čuvanje podataka
- **Scheduler** koristi podatke učitane iz fajla
- **LearningGraph** ažurira vrednosti koje se kasnije čuvaju

Na ovaj način modul predstavlja vezu između memorije programa i trajnog skladišta podataka.

2.3 Autocomplete i suggestions funkcionalnost

Ovu funkcionalnost obezbeđuju 2 modula projekta - implementacija prefiksnog drveta *Trie.h* i implementacija sistema određivanja predloga korišćenjem Levenshtein distance, u modulu *SuggestionHelper.h*.

Trie (prefiksno drvo) predstavlja strukturu podataka koja omogućava efikasno skladištenje i pretragu stringova po prefiksu. Osnovna prednost Trie strukture je vremenska složenost pretrage $O(k)$, gde je k dužina traženog prefiksa. Sve reči i fraze na engleskom jeziku ubacuju se u strukturu, a zatim se tokom unosa generišu kandidati za nastavak.

Na primer:

- unos: `go`
- rezultat: `good morning`, `good night`

2.3.1 Implementacija strukture `Trie`

Trie predstavlja prefiksno drvo koje se koristi za efikasnu pretragu stringova po početnom delu teksta. U okviru ovog sistema koristi se za autocomplete funkcionalnost nad engleskim rečima i frazama.

```
#ifndef TRIE_H
#define TRIE_H

#include <string>
#include <vector>
#include <unordered_map>
#include <algorithm>
#include <memory>

class TrieNode {
public:
    std::unordered_map<char, std::unique_ptr<TrieNode>> children;
    bool isEndOfWord = false;
};

class Trie {
private:
    std::unique_ptr<TrieNode> root;
```

```
    void collectWords(const TrieNode* node,
                      const std::string& current,
                      std::vector<std::string>& results,
                      int limit) const { }

public:
    Trie() : root(std::make_unique<TrieNode>()) {}

    void clear() {
        root = std::make_unique<TrieNode>();
    }

    void insert(const std::string& word) { }

    bool search(const std::string& word) const { }

    std::vector<std::string> getCandidatesByPrefix(const std::
       string& prefix, int limit = 20) const { }
};

#endif
```

Opis implementacije Trie se sastoji od čvorova tipa `TrieNode`. Svaki čvor sadrži mapu dece i indikator da li se u datom čvoru završava cela reč ili fraza.

Struktura koristi `unordered_map<char, unique_ptr<TrieNode»`, čime se omogućava fleksibilna podrška za karaktere, uključujući i razmake, što je važno zbog rada sa frazama.

Glavne funkcije

- `insert` – ubacuje novu reč ili frazu u prefiksno drvo
- `search` – proverava da li data reč postoji
- `getCandidatesByPrefix` – vraća listu kandidata za autocomplete
- `clear` – resetuje celu strukturu

Veza sa sistemom Trie se puni svim engleskim rečima i frazama iz baze, a zatim se koristi tokom unosa korisnika za generisanje autocomplete kandidata. Dobijeni kandidati se dalje prosleđuju modulu za suggestions i rangiranje.

2.3.2 Implementacija strukture `Suggestion`

Modul `SuggestionHelper` zadužen je za generisanje i rangiranje predloga (suggestions) kada korisnik unese netačan ili nepotpun odgovor. Za razliku od modula `Trie`, koji je zadužen za pronalaženje kandidata po prefiksu, `SuggestionHelper` vrši dodatnu obradu i procenu kvaliteta tih kandidata. Ovaj modul kombinuje više tehnika, uključujući Levenshtein distancu i heurističko rangiranje kandidata. Osnovna ideja implementacije jeste da se korisniku ne prikažu samo svi mogući kandidati, već da se kandidati sortiraju tako da najrelevantniji predlozi budu na vrhu liste.
Modul je implementiran kao klasa sa isključivo statičkim metodama. Ovakav pristup je odabran zato što klasa ne čuva sopstveno stanje, već služi samo kao skup funkcija koje obrađuju podatke prosleđene iz drugih delova sistema.

Na ovaj način:

- nije potrebno praviti objekte klase
- funkcije su lako dostupne iz GUI sloja i ostalih modula
- implementacija ostaje jednostavna i pregledna

Opis implementacije

```
#ifndef SUGGESTION_HELPER_H
#define SUGGESTION_HELPER_H

#include <string>
#include <vector>
#include <algorithm>
#include <limits>
#include "Word.h"

class SuggestionHelper {
private:
    static std::string toLowerCase(std::string s) { }

public:
    static int levenshteinDistance(const std::string& a, const
       std::string& b) {
        int n = (int)a.size();
        int m = (int)b.size();

        std::vector<std::vector<int>> dp(n + 1, std::vector<int>(
           m + 1, 0));
```

```
    for (int i = 0; i <= n; i++) dp[i][0] = i;
    for (int j = 0; j <= m; j++) dp[0][j] = j;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            int cost = (a[i - 1] == b[j - 1]) ? 0 : 1;

            dp[i][j] = std::min({
                dp[i - 1][j] + 1,
                dp[i][j - 1] + 1,
                dp[i - 1][j - 1] + cost
            });
        }
    }

    return dp[n][m];
}

static std::vector<std::string> rankAutocompleteSuggestions(
    const std::string& input,
    const std::vector<std::string>& candidates,
    const std::vector<Word>& words,
    int limit = 3
) {
    struct ScoredCandidate {
        std::string word;
        int wrongCount;
        int editDistance;
        int length;
        int score;
    };

    std::vector<ScoredCandidate> scored;
    std::string normalizedInput = toLowerCase(input);

    for (const auto& candidate : candidates) {
        int wrong = 0;

        for (const auto& w : words) {
```

```
            if (toLowerCase(w.english) == candidate) {
                wrong = w.wrongCount;
                break;
            }
        }

        int dist = levenshteinDistance(normalizedInput,
           candidate);
        int lenPenalty = (int)candidate.size() - (int)
           normalizedInput.size();
        if (lenPenalty < 0) lenPenalty = 0;

        int score = 5 * wrong - 2 * dist - lenPenalty;

        scored.push_back({
            candidate,
            wrong,
            dist,
            (int)candidate.size(),
            score
        });
    }

    std::sort(scored.begin(), scored.end(),
        [](const ScoredCandidate& a, const ScoredCandidate& b
           ) {
            if (a.score != b.score)
                return a.score > b.score;
            if (a.editDistance != b.editDistance)
                return a.editDistance < b.editDistance;
            if (a.wrongCount != b.wrongCount)
                return a.wrongCount > b.wrongCount;
            if (a.length != b.length)
                return a.length < b.length;
            return a.word < b.word;
        });

    std::vector<std::string> result;
    for (int i = 0; i < (int)scored.size() && i < limit; i++)
        {
```

```
            result.push_back(scored[i].word);
        }

        return result;
    }

    static std::vector<std::string> getTopFuzzySuggestions(
        const std::string& input,
        const std::vector<Word>& words,
        int limit = 3,
        int maxDistance = 2
    ) {
        struct FuzzyCandidate {
            std::string word;
            int distance;
            int wrongCount;
            int score;
        };

        std::vector<FuzzyCandidate> candidates;
        std::string normalizedInput = toLowerCase(input);

        for (const auto& w : words) {
            std::string candidate = toLowerCase(w.english);
            int dist = levenshteinDistance(normalizedInput,
               candidate);

            if (dist <= maxDistance) {
                int score = 5 * w.wrongCount - 3 * dist;

                candidates.push_back({
                    w.english,
                    dist,
                    w.wrongCount,
                    score
                });
            }
        }

        std::sort(candidates.begin(), candidates.end(),
```

```
            [](const FuzzyCandidate& a, const FuzzyCandidate& b)
               {
                if (a.score != b.score)
                    return a.score > b.score;
                if (a.distance != b.distance)
                    return a.distance < b.distance;
                if (a.wrongCount != b.wrongCount)
                    return a.wrongCount > b.wrongCount;
                return a.word < b.word;
            });

        std::vector<std::string> result;
        for (int i = 0; i < (int)candidates.size() && i < limit;
           i++) {
            result.push_back(candidates[i].word);
        }

        return result;
    }
};

#endif
```

Normalizacija unosa Prvi korak u implementaciji predstavlja funkcija `toLowerCase`. Njena uloga je da sve ulazne stringove prevede u mala slova.

Na taj način sistem postiže:

- neosetljivost na razliku između velikih i malih slova
- ujednačen način poređenja stringova
- stabilnije suggestions rezultate

Bez ovog koraka korisnik bi mogao uneti, na primer, `Apple` umesto `apple`, što bi nepotrebno dovelo do netačnog rezultata ili lošijeg rangiranja kandidata.

Levenshtein distanca Funkcija `levenshteinDistance` implementira algoritam za izračunavanje minimalnog broja operacija (ubacivanje, brisanje, zamena) potrebnih da se jedan string transformiše u drugi.

Ovaj algoritam koristi dinamičko programiranje i gradi matricu dimenzija $(n + 1) \times (m + 1)$, gde su n i m dužine posmatranih stringova.

Složenost algoritma je:

$$O(n \cdot m)$$

Ova funkcija omogućava detekciju grešaka u unosu korisnika (typos).

Za dva stringa dužine n i m formira se matrica dimenzija $(n+1) \times (m+1)$, gde element `dp[i][j]` predstavlja minimalan broj operacija potrebnih da se prvih i karaktera prvog stringa transformišu u prvih j karaktera drugog stringa.

U implementaciji se najpre inicijalizuju granični slučajevi:

- `dp[i][0] = i`, što znači da je za prelazak sa stringa dužine i na prazan string potrebno i brisanja
- `dp[0][j] = j`, što znači da je za prelazak sa praznog stringa na string dužine j potrebno j ubacivanja

Nakon toga matrica se popunjava po sledećoj logici:

- ako su trenutni karakteri jednaki, cena zamene je 0
- ako nisu jednaki, cena zamene je 1
- zatim se bira minimum između:
  - brisanja
  - ubacivanja
  - zamene

Konačna vrednost `dp[n][m]` predstavlja Levenshtein distancu između dva posmatrana stringa.

Na ovaj način implementacija omogućava sistemu da proceni koliko je korisnikov unos udaljen od očekivane reči ili fraze.

Rangiranje autocomplete kandidata Funkcija `rankAutocompleteSuggestions` prima listu kandidata dobijenih iz Trie strukture i dodatno ih rangira.

Za svaki kandidat računa se:

- Levenshtein distanca
- broj prethodnih grešaka korisnika
- razlika u dužini između unosa i kandidata

Na osnovu toga formira se heuristički score:

$$score = 5 \cdot wrongCount - 2 \cdot distance - lengthPenalty$$

Kandidati se zatim sortiraju prema:

- najvećem score-u
- najmanjoj distanci
- najvećem broju grešaka

Na ovaj način sistem daje prioritet rečima koje su:

- slične unetom tekstu
- problematične za korisnika

Implementacija ove funkcije sastoji se iz nekoliko koraka.

1. Definisanje pomoćne strukture
   U okviru funkcije definisana je pomoćna lokalna struktura `ScoredCandidate`, koja čuva:
   - samu reč kandidata
   - broj prethodnih grešaka korisnika za tu reč
   - Levenshtein distancu između korisničkog unosa i kandidata
   - dužinu kandidata
   - ukupni heuristički score

   Razlog za uvođenje ove strukture jeste to što sistem ne rangira kandidate samo po jednom kriterijumu, već po više međusobno povezanih faktora.
2. Normalizacija korisničkog unosa
   Uneti tekst se najpre prevodi u mala slova pomoću funkcije `toLowerCase`. Time se obezbeđuje da se svi kandidati porede nad istim oblikom teksta.
3. Obrada svakog kandidata
   Za svakog kandidata iz liste vrši se sledeća obrada:
   - pronalazi se koliko je puta korisnik ranije grešio tu reč
   - računa se Levenshtein distanca u odnosu na korisnički unos

- računa se kazna zbog razlike u dužini između unosa i kandidata
- računa se konačni score

Broj grešaka se dobija prolaskom kroz vektor `words`. Kada se pronađe reč čiji engleski tekst odgovara kandidatu, preuzima se njena vrednost `wrongCount`.

Ovaj korak je važan zato što suggestions nisu zasnovane samo na tekstualnoj sličnosti, već i na prethodnom ponašanju korisnika. Time sistem dobija personalizovani karakter.

4. Heuristički score
   Score kandidata računa se formulom:

$$score = 5 \cdot wrongCount - 2 \cdot editDistance - lengthPenalty$$

   Ova formula uvedena je kao heuristika koja kombinuje više kriterijuma:

   - veći broj grešaka povećava score, jer sistem želi da prioritet da rečima koje su korisniku problematične
   - veća edit distanca smanjuje score, jer udaljeniji kandidati nisu dobri predlozi
   - veća razlika u dužini dodatno umanjuje score, jer veoma duge reči često nisu prirodan nastavak unosa

   Drugim rečima, najbolji kandidat je onaj koji je:

   - dovoljno sličan korisničkom unosu
   - relativno blizu po dužini
   - važan za korisnika zato što ga često greši

5. Sortiranje kandidata
   Kada se za sve kandidate izračunaju svi potrebni atributi, kandidati se sortiraju pomoću `std::sort`.

   Redosled prioriteta u sortiranju je:

   (a) najveći score
   (b) najmanja edit distanca
   (c) najveći broj grešaka
   (d) najmanja dužina reči
   (e) leksikografski redosled

Ovaj redosled je pažljivo odabran:

- score predstavlja glavni kriterijum
- edit distanca služi kao dodatno preciziranje
- broj grešaka dodatno pojačava personalizaciju
- kraće reči imaju blagu prednost jer su češće prirodniji nastavak korisničkog unosa
- leksikografski redosled koristi se samo kao poslednji stabilizacioni kriterijum

6. Ograničavanje broja rezultata
   Nakon sortiranja uzimaju se samo prva `limit` kandidata. Time sistem izbegava prevelik broj predloga i korisniku nudi samo mali broj najboljih opcija. U ovom konkretnom slučaju, najčešće se koristi limit 3.

Fuzzy suggestions Funkcija `getTopFuzzySuggestions` koristi se kada ne postoje dobri prefiksni kandidati.

Za svaku reč iz baze:

- računa se Levenshtein distanca
- zadržavaju se samo reči unutar zadatog praga (`maxDistance`)

Zatim se vrši rangiranje prema:

- score-u
- distanci
- broju grešaka

To znači da u ovom slučaju sistem ne polazi od kandidata dobijenih iz Trie-a, već razmatra sve reči iz baze i pokušava da pronađe one koje su dovoljno slične korisnikovom unosu.

Kao i kod autocomplete rangiranja, uvodi se pomoćna struktura `FuzzyCandidate`, koja čuva:

- samu reč
- Levenshtein distancu
- broj grešaka korisnika za tu reč
- ukupan score

I ovde se korisnički unos najpre prevodi u mala slova. Za svaku reč u vektoru `words` radi se sledeće:

- uzima se engleski tekst reči
- prevodi se u mala slova
- računa se Levenshtein distanca u odnosu na unos

Za razliku od prethodne funkcije, ovde se ne oslanja na već izabrane prefiksne kandidate, nego se pregledaju sve reči. Reč se uzima u obzir samo ako važi:

$$dist \leq maxDistance$$

U implementaciji je tipično `maxDistance = 2`. To znači da sistem neće nuditi potpuno nepovezane reči, već samo one koje su dovoljno bliske korisničkom unosu. Za fuzzy suggestions koristi se formula:

$$score = 5 \cdot wrongCount - 3 \cdot distance$$

Ovde je logika slična:

- reči koje je korisnik ranije grešio dobijaju veći prioritet
- veća distanca umanjuje relevantnost kandidata

Time sistem i u fuzzy režimu ostaje personalizovan. Kandidati se dalje sortiraju po:

1. najvećem score-u
2. najmanjoj distanci
3. najvećem broju grešaka
4. leksikografskom redosledu

Nakon toga uzima se najviše `limit` kandidata i oni se vraćaju GUI sloju radi prikaza korisniku.

Uloga u sistemu Modul `SuggestionHelper` povezuje više komponenti sistema:

- prima kandidate iz **Trie** strukture
- koristi podatke iz **Word** modela

- koristi se u **GUI** za prikaz predloga

Ovaj modul predstavlja važan deo sistema zato što omogućava:

- inteligentnu obradu netačnih unosa
- personalizaciju suggestions mehanizma
- kombinovanje tekstualne sličnosti i korisničke istorije

Na taj način korisnik ne dobija samo tehnički tačne predloge, već predloge koji su prilagođeni njegovom prethodnom radu i nivou znanja. Takođe, omogućava inteligentno ponašanje sistema prilikom unosa korisnika i značajno poboljšava korisničko iskustvo.

2.4 `Scheduler`

Modul `Scheduler` predstavlja centralni deo sistema zadužen za izbor sledeće reči ili fraze za učenje. Za razliku od jednostavnog sekvencijalnog ili nasumičnog prikaza, ovaj sistem koristi više parametara kako bi odredio koja stavka trenutno ima najveći prioritet.

```
#ifndef SCHEDULER_H
#define SCHEDULER_H

#include <vector>
#include <ctime>
#include <limits>
#include "Word.h"

class Scheduler {
private:
    static int calculatePriority(const Word& word) { }

public:
    static bool hasReadyWords(const std::vector<Word>& words) { }

    static int getNextWordIndex(const std::vector<Word>& words) {
        }
};

#endif
```

2.4.1 Opis implementacije

Klasa `Scheduler` implementirana je kao skup statičkih metoda. Ovakav pristup je odabran zato što modul ne mora da čuva sopstveno stanje, već isključivo vrši obradu nad kolekcijom objekata tipa `Word`.

Funkcija `calculatePriority` Privatna funkcija `calculatePriority` računa prioritet jedne reči. Prioritet predstavlja numeričku procenu toga koliko je određena reč važna za sledeće pojavljivanje.

Implementacija koristi sledeće podatke:

- broj pogrešnih odgovora (`wrongCount`)
- trenutnu težinu reči (`learningWeight`)
- nivo usvojenosti (`masteryLevel`)
- vreme sledeće provere (`nextReviewTime`)

Najpre se računa `overdueBonus`, odnosno bonus za reči koje su već dospele za review i kasne u odnosu na trenutno vreme. Ovaj bonus se izražava kao broj dana kašnjenja.

Nakon toga formira se heuristička funkcija prioriteta:

$$priority = 10 \cdot wrongCount + 4 \cdot learningWeight - 5 \cdot masteryLevel + overdueBonus$$

Ova formula implementirana je tako da:

- veći broj grešaka značajno poveća prioritet
- veća težina reči dodatno poveća prioritet
- viši nivo usvojenosti smanji prioritet
- kašnjenje u review-u poveća prioritet

Funkcija `hasReadyWords` Funkcija `hasReadyWords` proverava da li postoji makar jedna reč koja je spremna za review. Reč se smatra spremnom ako važi:

$$nextReviewTime \leq now$$

Ova funkcija je korisna za proveru da li sesija učenja uopšte može da se nastavi.

Funkcija `getNextWordIndex` Funkcija `getNextWordIndex` predstavlja glavnu funkciju modula. Njena uloga je da pronađe indeks reči sa najvećim prioritetom među svim rečima koje su trenutno spremne za review.

Implementacija radi u sledećim koracima:

1. uzima trenutno vreme
2. prolazi kroz sve reči u vektoru
3. razmatra samo one reči čije je `nextReviewTime` manje ili jednako trenutnom vremenu
4. za svaku takvu reč računa prioritet
5. pamti indeks reči sa najvećim prioritetom

Ako nijedna reč nije spremna, funkcija vraća `-1`.

2.4.2 Veza sa ostatkom sistema

Scheduler koristi podatke iz strukture `Word`, koje prethodno ažurira modul `LearningGraph`. Na osnovu tih podataka određuje sledeću reč koja će biti prikazana u GUI sloju.

Na taj način ovaj modul predstavlja mehanizam koji spaja:

- podatke o prethodnim greškama korisnika
- model napretka
- vreme sledećeg ponavljanja

2.4.3 Značaj modula

Bez ovog modula sistem bi mogao samo da prikazuje reči redom ili nasumično. Uvođenjem `Scheduler`-a omogućena je personalizacija, jer se prioritet daje onim rečima koje su korisniku problematične ili koje su upravo dospele za ponovno ponavljanje.

2.5 `LearningGraph.h`

Modul `LearningGraph` implementira model napretka korisnika kroz više diskretnih stanja znanja. Za razliku od jednostavnog binarnog pristupa, gde je odgovor samo tačan ili netačan, ovde se uvodi više nivoa usvojenosti reči.

```
#ifndef LEARNING_GRAPH_H
#define LEARNING_GRAPH_H

#include <string>
#include <vector>
#include <iostream>
#include <algorithm>

class LearningGraph {
private:
    std::vector<std::string> levelNames = {
        "New",
        "Learning",
        "Familiar",
        "Mastered"
    };

public:
    std::string getLevelName(int level) const { }

    int advanceOnCorrect(int currentLevel) const { }

    int degradeOnWrong(int currentLevel) const { }

    int updateWeightOnCorrect(int currentWeight) const { }

    int updateWeightOnWrong(int currentWeight) const { }

    void printGraph() const { }
};

#endif
```

2.5.1 Opis implementacije

Klasa `LearningGraph` modeluje proces učenja kao skup stanja kroz koja reč prolazi tokom rada sistema. Stanja su definisana preko vektora `levelNames`, koji sadrži četiri nivoa:

- New

- Learning
- Familiar
- Mastered

Funkcija `getLevelName` Funkcija `getLevelName` koristi se za pretvaranje numeričkog nivoa u tekstualni naziv. Time GUI i drugi delovi sistema mogu prikazati korisniku smislen opis stanja, umesto samog broja. Implementacija sadrži i zaštitu od neispravnih vrednosti nivoa, pri čemu se u tom slučaju vraća tekst `Unknown`.

Funkcija `advanceOnCorrect` Ova funkcija određuje prelaz između nivoa nakon tačnog odgovora. Implementacija povećava nivo za jedan, osim ako je reč već na maksimalnom nivou `Mastered`. Na taj način dobijaju se sledeći prelazi:

- New → Learning
- Learning → Familiar
- Familiar → Mastered
- Mastered → Mastered

Funkcija `degradeOnWrong` Ova funkcija određuje prelaz nakon pogrešnog odgovora. Implementacija smanjuje nivo za jedan, osim ako je reč već na minimalnom nivou `New`. Time se dobijaju sledeći prelazi:

- Mastered → Familiar
- Familiar → Learning
- Learning → New
- New → New

Funkcije za ažuriranje težine Pored nivoa znanja, sistem održava i `learningWeight`, koji predstavlja procenu težine reči za korisnika.

Kod tačnog odgovora funkcija `updateWeightOnCorrect` smanjuje težinu za jedan, uz donju granicu 1:

$$weight = \max(1, weight - 1)$$

Kod pogrešnog odgovora funkcija `updateWeightOnWrong` povećava težinu za dva, uz gornju granicu 10:

$$weight = \min(10, weight + 2)$$

Na ovaj način sistem posebno prati koliko je reč problematična za korisnika.

Funkcija `printGraph` Funkcija `printGraph` služi za ispis modela učenja u tekstualnom obliku. Ona prikazuje sve prelaze između nivoa i pravila za ažuriranje težine. Ova funkcija je korisna za demonstraciju rada sistema i za proveru logike modela. Korišćena je fazi testiranja programa dok je bio u razvoju i bez GUI-a.

2.5.2 Veza sa ostatkom sistema

Modul `LearningGraph` koristi se zajedno sa drugim delovima sistema:

- `Word` čuva nivo i težinu reči
- `GUI` poziva funkcije ovog modula nakon svakog odgovora korisnika
- `Scheduler` koristi ažurirane vrednosti nivoa i težine pri izboru sledeće reči

Na taj način ovaj modul predstavlja logički model promene stanja učenja i uvodi element personalizacije u sistem.

2.6 `RabinKarp.h`

Modul `RabinKarp` implementira Rabin–Karp algoritam za pretragu podstringa. U okviru sistema koristi se za proveru da li se očekivani odgovor nalazi unutar korisničkog tekstualnog unosa.

```
#ifndef RABIN_KARP_H
#define RABIN_KARP_H

#include <string>

class RabinKarp {
private:
    static const int d = 256;   // broj karaktera
    static const int q = 101;   // prost broj

public:
    static bool search(const std::string& text, const std::string
       & pattern) { }
};

#endif
```

2.6.1 Opis implementacije

Klasa `RabinKarp` implementirana je kao klasa sa statičkom funkcijom `search`, jer algoritam ne zahteva čuvanje unutrašnjeg stanja između poziva.

Konstante `d` i `q` Konstanta `d` predstavlja broj mogućih karaktera i koristi se kao baza pri izračunavanju hash vrednosti. U implementaciji je postavljena na 256.

Konstanta `q` predstavlja prost broj koji se koristi kao modul. Njegova uloga je da ograniči rast hash vrednosti i smanji broj kolizija.

Funkcija `search` Funkcija `search` prima dva stringa:

- `text` – korisnički unos
- `pattern` – očekivani odgovor

Najpre se proverava da li je obrazac duži od teksta. Ako jeste, obrazac se ne može nalaziti unutar teksta i funkcija odmah vraća `false`.

Zatim se uvode promenljive:

- `p` – hash vrednost obrasca
- `t` – hash vrednost trenutnog prozora u tekstu
- `h` – pomoćna vrednost za uklanjanje prvog karaktera iz rolling hash izraza

Najpre se računa vrednost:

$$h = d^{m-1}q$$

Nakon toga izračunavaju se inicijalni hash-evi obrasca i prvog prozora u tekstu.

Poređenje hash vrednosti Za svaki prozor dužine `m` u tekstu proverava se da li su hash vrednosti obrasca i prozora jednake. Ako jesu, vrši se dodatna provera karakter po karakter, kako bi se izbegli problemi zbog mogućih kolizija.

Ako se svi karakteri poklapaju, funkcija vraća `true`.

Rolling hash Ako trenutni prozor nije poslednji, hash prozora se ažurira bez ponovnog računanja od početka. To se postiže uklanjanjem doprinosa prvog karaktera, pomeranjem hash vrednosti i dodavanjem narednog karaktera.

Na taj način implementira se rolling hash mehanizam, koji predstavlja centralnu ideju Rabin–Karp algoritma.

2.6.2 Veza sa ostatkom sistema

Modul `RabinKarp` koristi se u GUI sloju prilikom provere odgovora korisnika. Njegova uloga je da omogući fleksibilniji unos, tako da korisnik može uneti i celu rečenicu, a da sistem ipak prepozna prisustvo tačne reči ili fraze.

Na primer:

- očekivani odgovor: `good morning`
- korisnički unos: `I would say good morning`

U ovom slučaju odgovor se prihvata kao tačan. Uvođenje Rabin–Karp algoritma omogućava sistemu:

- fleksibilniju proveru odgovora
- podršku za fraze
- primenu algoritma za pretragu stringova na smislen način

Na taj način ovaj modul predstavlja važan deo sistema i uvodi dodatni nivo inteligencije pri obradi korisničkog unosa.

3 Korisnički interfejs

3.1 Način korišćenja aplikacije

Glavni ulaz u sistem predstavlja grafički korisnički interfejs implementiran u fajlu `gui.cpp`. Ovaj deo projekta omogućava korisniku da na jednostavan i pregledan način komunicira sa sistemom za učenje reči i fraza.

3.1.1 Pokretanje aplikacije

Pri pokretanju se najpre učitavaju podaci iz JSON fajlova, a zatim se prikazuje glavni prozor aplikacije. Izgled aplikacije može se videti na slici ispod.

U ovom koraku sistem:

- učitava bazu reči i fraza
- učitava ranije sačuvan napredak korisnika
- formira internu strukturu za autocomplete i suggestions
- prikazuje prvu reč ili frazu spremnu za učenje

Na ovaj način korisnik odmah po pokretanju može da nastavi rad bez dodatne konfiguracije.

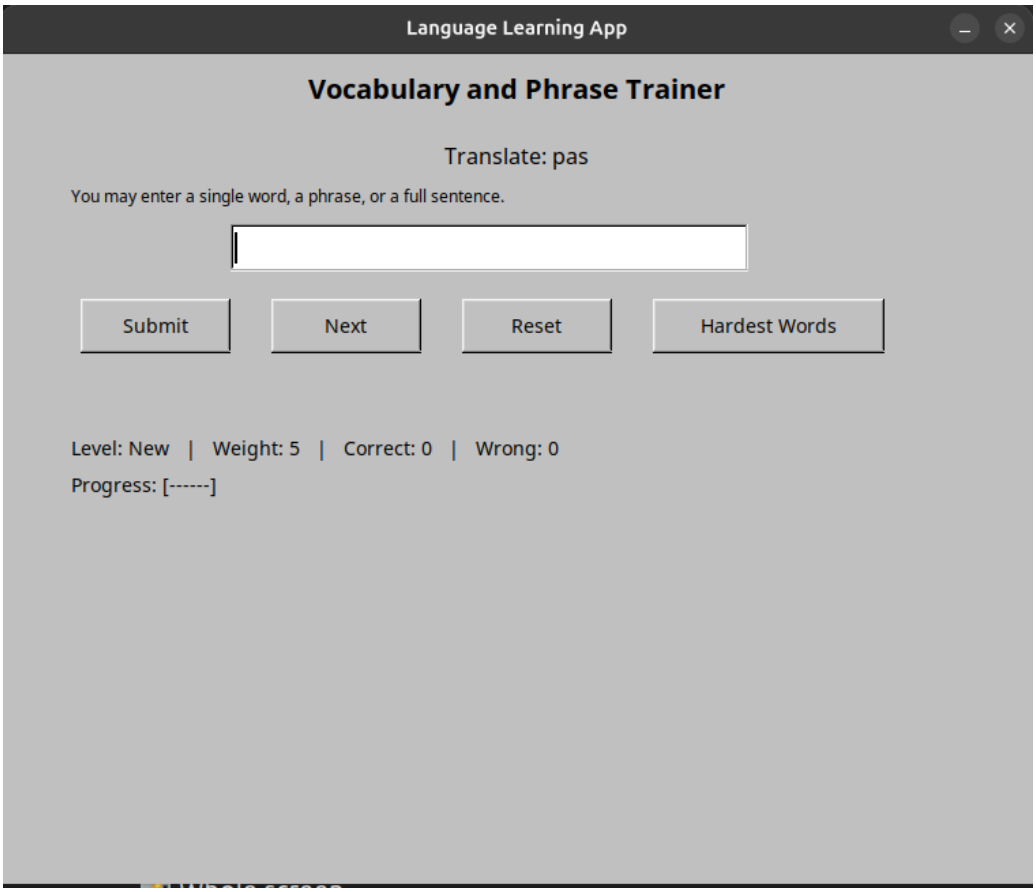


Figure 1: Izgled prozora pri pokretanju aplikacije

3.1.2 Uloga fajla `words.json`

Fajl `words.json` predstavlja početnu bazu podataka sistema. U njemu se nalaze sve reči i fraze koje aplikacija koristi tokom rada.

Svaki zapis u ovom fajlu sadrži:

- srpski tekst
- odgovarajući engleski prevod

Primer sadržaja baze se može videti na slici `Figure 2`.

Iz korisničkog ugla, `words.json` predstavlja skup svih reči koje aplikacija može zadati korisniku za vežbanje. Kada se sistem resetuje, aplikacija se vraća upravo na stanje definisano u ovom fajlu.

3.1.3 Uloga fajla `progress.json`

Fajl `progress.json` služi za čuvanje napretka korisnika. Za razliku od `words.json`, koji sadrži samo početne podatke, ovaj fajl čuva dinamičke informacije koje nastaju tokom rada.

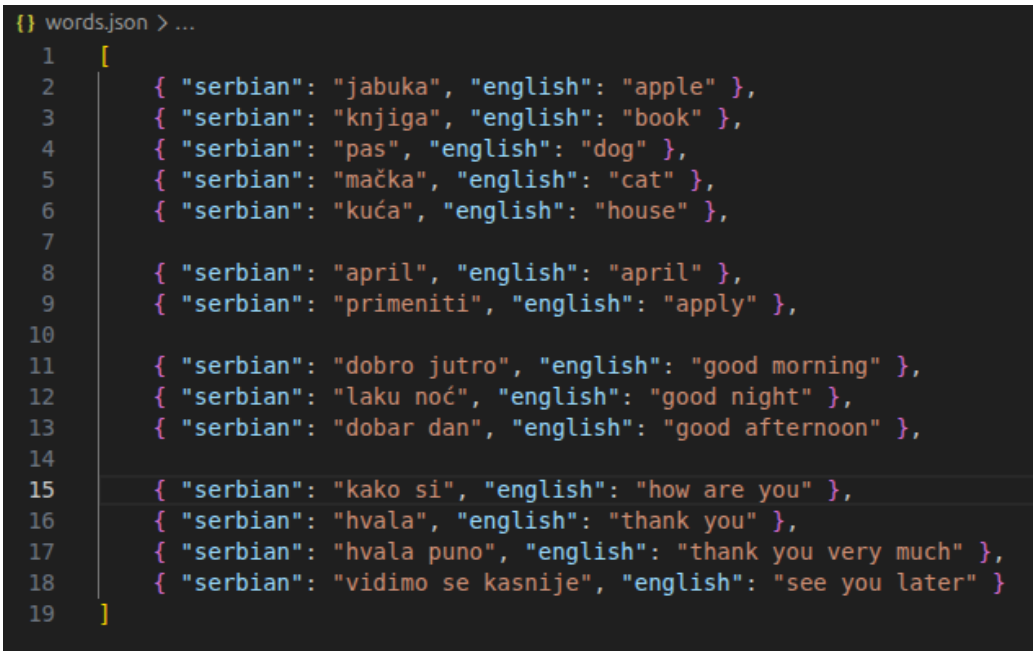


Figure 2: Sadržaj datoteke words.json

U njemu se za svaku reč ili frazu beleži:

- broj tačnih odgovora
- broj pogrešnih odgovora
- nivo usvojenosti
- težina reči
- interval ponavljanja
- sledeće vreme za review

Iz korisničkog ugla, `progress.json` omogućava da aplikacija „pamti“ prethodni rad. Zahvaljujući tome, korisnik nakon ponovnog pokretanja sistema ne počinje od nule, već nastavlja tamo gde je stao. Izgled datoteke progress.json, nakon završetka 1 sesije, može se videti na slici `Figure 3`.



Figure 3: Izgled aplikacije nakon što je korisnik uneo tačan odgovor

3.1.4 Osnovni tok korišćenja

Korišćenje aplikacije odvija se kroz jednostavan ciklus:

1. sistem prikaže reč ili frazu na srpskom jeziku
2. korisnik u polje za unos upisuje engleski prevod
3. korisnik klikne dugme `Submit`
4. aplikacija proverava odgovor i prikazuje rezultat
5. korisnik po potrebi koristi suggestions ili prelazi na sledeću reč

Na ovaj način korisničko iskustvo ostaje jednostavno i intuitivno.

3.1.5 Unos odgovora

Korisnik može uneti:

- jednu reč
- celu frazu
- čak i celu rečenicu koja sadrži tačan odgovor

Ovo je posebno korisno kod fraza, jer sistem ne zahteva strogo da unos bude identičan samo jednoj izolovanoj reči. Zahvaljujući tome, aplikacija je fleksibilnija i prirodnija za korišćenje.

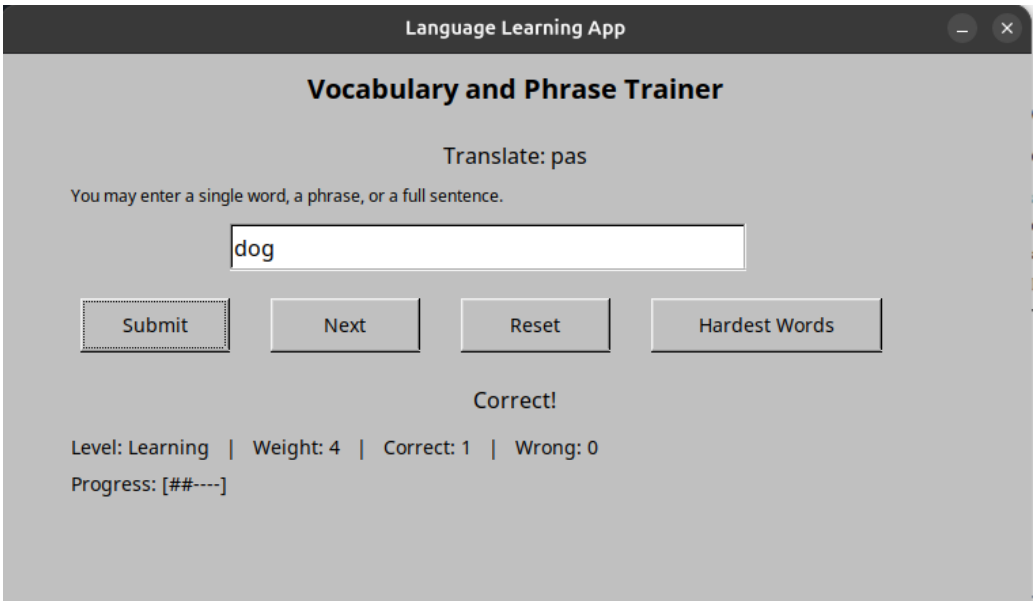


Figure 4: Izgled aplikacije nakon što je korisnik uneo tačan odgovor

U oba slučaja sistem može prepoznati prisustvo tačne fraze. Ovu funkcionalnost omogućava modul RabinKarp.h.

3.1.6 Dugme `Submit`

Dugme `Submit` predstavlja osnovnu akciju u aplikaciji. Kada korisnik klikne ovo dugme, sistem proverava uneti odgovor.

Rezultat može biti:

- `Correct!` – ako je odgovor tačan
- `Correct (expected answer found in sentence)!` – ako se tačan odgovor pojavljuje unutar dužeg unosa
- `Wrong!` – ako odgovor nije tačan

Nakon ove provere sistem ažurira korisnikov napredak i snima nove vrednosti u `progress.json`.

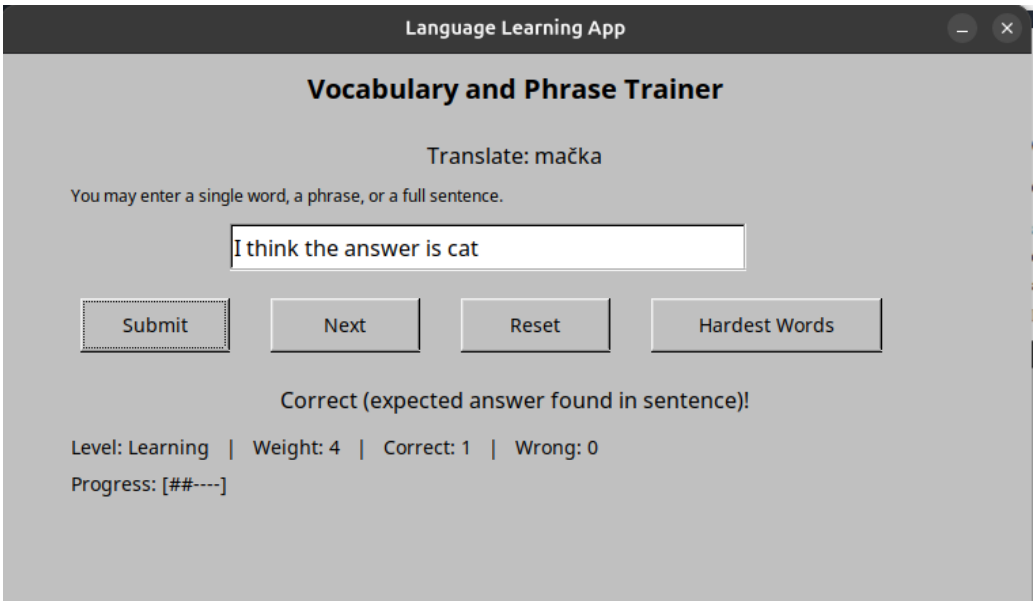


Figure 5: Izgled aplikacije nakon što je korisnik uneo tačan odgovor, fleksibilni unos, primena Rabin Karpa

3.1.7 Prikaz statistike i napretka

Nakon svakog odgovora korisniku se prikazuju dodatne informacije o trenutno zadatoj reči ili frazi.

Prikazuju se:

- nivo usvojenosti (`Level`)
- težina reči (`Weight`)
- broj tačnih odgovora (`Correct`)
- broj pogrešnih odgovora (`Wrong`)
- vizuelni progress bar

Ovaj prikaz omogućava korisniku da vidi kako se njegovo znanje menja tokom rada. Na taj način sistem ne daje samo informaciju da li je odgovor tačan, već i širu sliku o stanju učenja.

3.1.8 Suggestions i pomoć korisniku

Ako korisnik unese netačan odgovor, sistem može ponuditi suggestions, odnosno predloge najverovatnijih tačnih odgovora.

Suggestions su korisne u sledećim situacijama:

- kada korisnik napravi malu slovnu grešku - ovaj deo omogućava SuggestionHelper.h

- kada unese samo deo reči - ovaj deo omogućava Trie.h
- kada je odgovor sličan ispravnom, ali nije potpuno tačan - SuggestionHelper.h

Na taj način aplikacija ne funkcioniše samo kao sistem za proveru, već i kao sistem za pomoć u učenju.

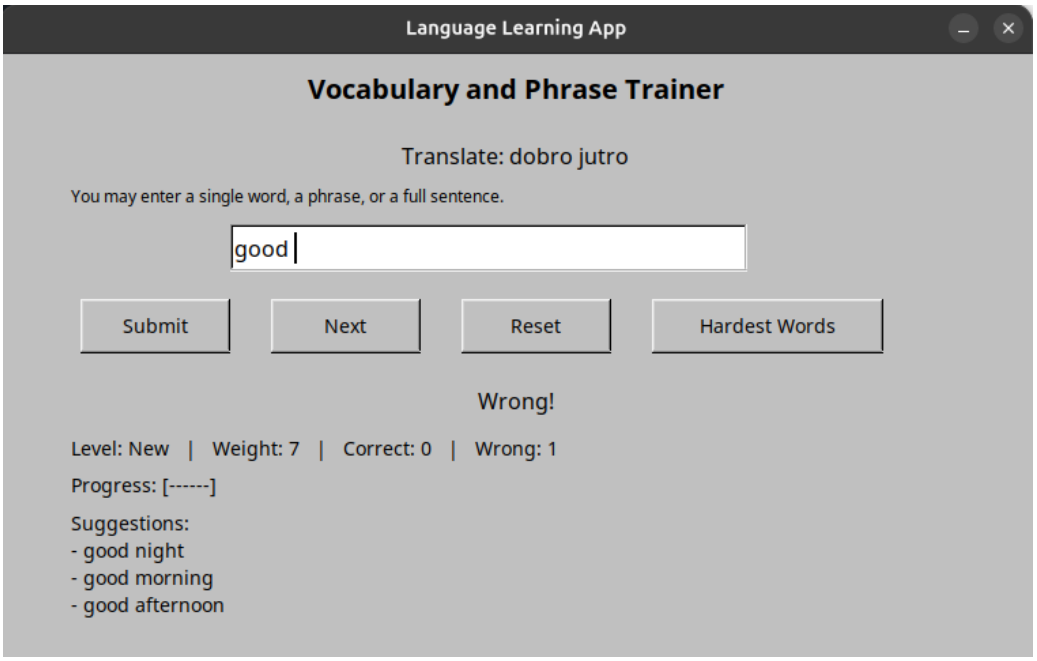


Figure 6: Izgled aplikacije nakon što je korisnik uneo nepotpun odgovor, autocomplete

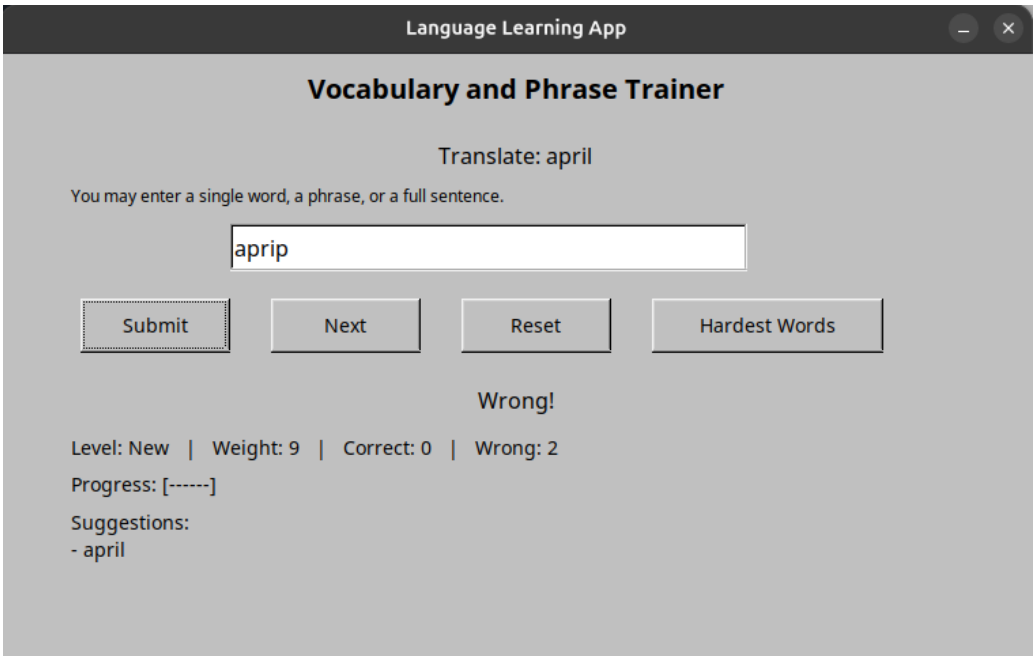


Figure 7: Izgled aplikacije nakon što je korisnik uneo pogrešan odgovor, typo

3.1.9 Dugme `Next`

Dugme `Next` omogućava prelazak na sledeću reč ili frazu. Nakon što korisnik vidi rezultat i eventualne suggestions, može kliknuti na ovo dugme i nastaviti rad.

Ovaj pristup je korisnički pogodan, jer korisnik najpre dobija vreme da pročita rezultat, a tek zatim samostalno prelazi na naredni zadatak.

3.1.10 Dugme `Reset`

Dugme `Reset` vraća aplikaciju na početno stanje definisano u fajlu `words.json`.

Iz korisničkog ugla, to znači:

- brišu se prethodno sačuvane statistike
- svi nivoi i težine se vraćaju na početne vrednosti
- sistem ponovo kreće od početne baze reči

Ova opcija je korisna u situacijama kada korisnik želi da počne učenje od početka ili kada želi da demonstrira sistem od nule.

3.1.11 Dugme `Hardest Words`

Dugme `Hardest Words` prikazuje korisniku listu najtežih reči i fraza.

U toj listi se prikazuju one stavke koje su se pokazale kao problematične, na primer:

- reči sa većim brojem grešaka

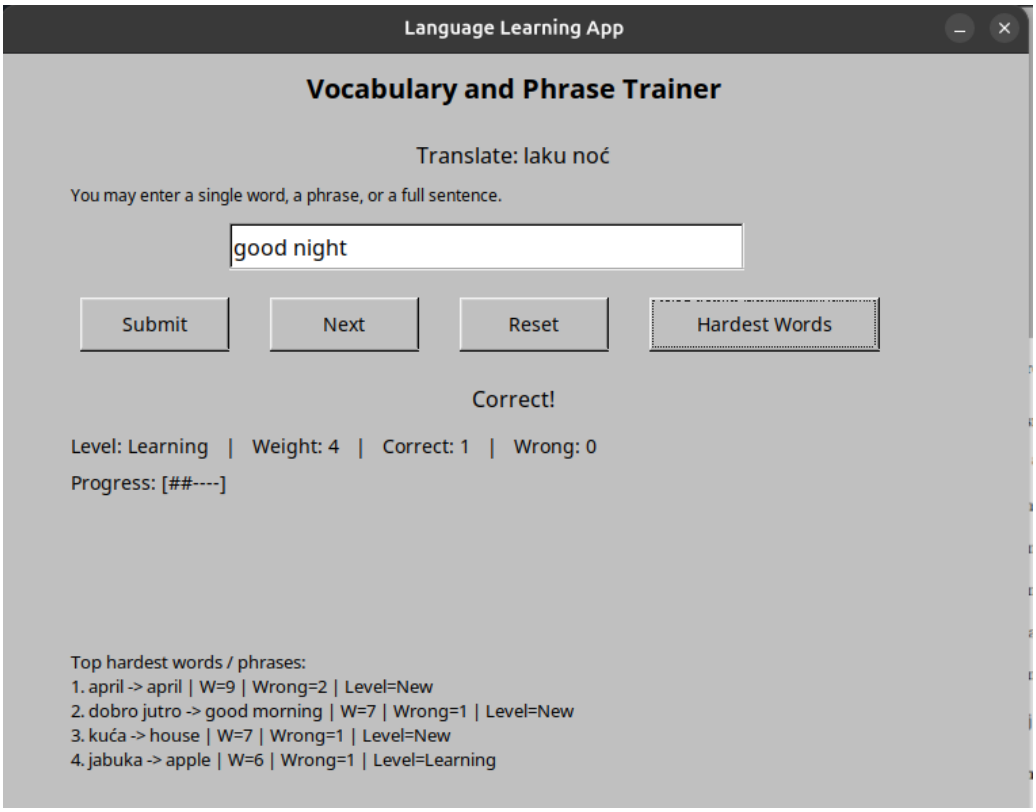


Figure 8: Izgled aplikacije nakon što je korisnik pritisnuo dugme Hardest Words

- reči sa većom težinom
- reči koje još nisu dovoljno savladane

Iz korisničkog ugla, ova funkcionalnost omogućava pregled onih reči i fraza kojima treba posvetiti dodatnu pažnju.

3.1.12 Način čuvanja napretka tokom rada

Jedna od važnih osobina sistema jeste to što se napredak ne čuva samo privremeno tokom jedne sesije. Nakon svakog odgovora aplikacija ažurira informacije o učenju i snima ih u `progress.json`.

To znači da korisnik može:

- zatvoriti aplikaciju
- kasnije je ponovo pokrenuti
- nastaviti rad od prethodnog stanja

Na ovaj način sistem oponaša ponašanje realnih aplikacija za učenje, koje ne zahtevaju da korisnik svaki put počinje od nule. Napredak se prikazuje i vizuelno kroz jednostavan progress bar koji odražava nivo znanja.

3.1.13 Značaj GUI sloja

Iako je najveći deo algoritamske logike implementiran u drugim modulima, `gui.cpp` ima ključnu ulogu jer sve te funkcionalnosti približava korisniku.

GUI sloj omogućava:

- jednostavno korišćenje sistema
- pregled rezultata u realnom vremenu
- intuitivno upravljanje aplikacijom
- jasnu vizuelizaciju napretka

Na taj način grafički interfejs predstavlja završni sloj koji spaja sve ostale delove projekta u jedinstvenu aplikaciju za personalizovano učenje.

4 Zaključak i moguća unapređenja sistema

U okviru ovog rada razvijen je sistem za učenje reči i fraza koji kombinuje više algoritamskih i softverskih koncepata u jedinstvenu aplikaciju. Sistem omogućava korisniku da kroz interaktivan proces uči prevode, dobija povratnu informaciju i prati sopstveni napredak tokom vremena.

Jedna od ključnih karakteristika sistema jeste personalizacija. Umesto da se sve reči tretiraju jednako, sistem uzima u obzir prethodne greške korisnika, nivo usvojenosti i vremenski raspored ponavljanja. Na taj način postiže se adaptivno ponašanje, gde se prioritet daje onim rečima koje su korisniku najproblematičnije.

U implementaciji su korišćeni različiti koncepti:

- Trie struktura za efikasno pronalaženje kandidata po prefiksu
- Levenshtein distanca za detekciju sličnosti između stringova
- Rabin–Karp algoritam za fleksibilnu proveru odgovora unutar dužeg teksta
- heuristički model rangiranja predloga
- model učenja zasnovan na diskretnim stanjima (LearningGraph)
- scheduler za izbor sledeće reči na osnovu prioriteta

Svi ovi elementi povezani su kroz grafički korisnički interfejs, koji omogućava jednostavno i intuitivno korišćenje aplikacije. Dodatno, sistem koristi JSON fajlove za čuvanje baze reči i napretka korisnika, čime se omogućava kontinuitet rada između različitih sesija.

Iako sistem u trenutnom obliku ispunjava osnovne funkcionalnosti i demonstrira kombinaciju više algoritamskih pristupa, postoji prostor za dalja unapređenja.

4.1 Moguća unapređenja

Jedno od potencijalnih unapređenja odnosi se na napredniji model rasporeda ponavljanja. Trenutna implementacija koristi jednostavnu heuristiku, dok bi se u budućnosti mogao primeniti složeniji spaced repetition model, poput SM-2 algoritma koji koriste aplikacije kao što je Anki.

Dalje unapređenje može se postići uvođenjem boljeg sistema za obradu prirodnog jezika. Trenutno se koristi Levenshtein distanca i Rabin–Karp algoritam, ali bi se mogli dodati:

- sinonimi i alternativni prevodi
- tolerancija na gramatičke varijacije
- podrška za više jezika

Takođe, korisnički interfejs bi mogao biti dodatno unapređen:

- moderniji vizuelni dizajn
- grafički prikaz statistike kroz dijagrame
- filtriranje i sortiranje reči po različitim kriterijumima

Sa aspekta sistema, moguće je uvesti:

- korisničke profile i više korisnika
- čuvanje podataka u bazi umesto JSON fajlova
- sinhronizaciju podataka između uređaja

Na kraju, sistem bi se mogao proširiti i na druge domene učenja, ne samo na reči i fraze, već i na:

- pojmove iz različitih oblasti
- pitanja i odgovore
- kompleksnije zadatke za učenje

4.2 Završna razmatranja

Razvijeni sistem pokazuje kako se kombinacijom osnovnih struktura podataka, algoritama i jednostavnog korisničkog interfejsa može implementirati funkcionalna aplikacija za učenje. Iako je sistem pojednostavljen u odnosu na komercijalna rešenja, on uspešno demonstrira ključne principe adaptivnog učenja i predstavlja dobru osnovu za dalji razvoj.